

A): FIFO PAGE REPLACEMENT

C PROGRAM:

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];

    int n, f, i, j, k = 0;

    int fault = 0, found;

    printf("Enter Number of Pages: ");

    scanf("%d", &n);

    printf("Enter Reference String:\n");

    for(i = 0; i < n; i++)

        scanf("%d", &pages[i]);

    printf("Enter Number of Frames: ");

    scanf("%d", &f);

    for(i = 0; i < f; i++)

        frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        found = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;

                break;
            }
        }
    }
}
```

```
}  
if(found == 0)  
{  
frames[k] = pages[i];  
k = (k + 1) % f;  
fault++;  
}  
}  
printf("Total Page Faults = %d\n", fault);  
return 0;  
}
```

SHELL SCRIPTING:

```
#!/bin/bash  
  
echo "FIFO Page Replacement Demonstration"  
  
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)  
  
frames=3  
  
echo "Reference String: ${pages[@]}"  
  
echo "Frames: $frames"  
  
echo "FIFO Algorithm Executed"
```

OUTPUT:

```
(kali㉿ Anbuchelvan)-[~]
$ nano Fifo.sh

(kali㉿ Anbuchelvan)-[~]
$ chmod +x Fifo.sh

(kali㉿ Anbuchelvan)-[~]
$ bash Fifo.sh
FIFO Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames: 3
FIFO Algorithm Executed
```

B) : LRU PAGE REPLACEMENT

C PROGRAM:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j;
    int fault = 0, count = 0;
    int found, pos, min;
    printf("Enter Number of Pages: ");
    scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++)
```

```
frames[i] = -1;
for(i = 0; i < n; i++)
{
    found = 0;
    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            count++;
            time[j] = count;
            found = 1;
            break;
        }
    }
    if(found == 0)
    {
        min = time[0];
        pos = 0;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }
        }
        if(time[pos] < min)
        {
            min = time[pos];
        }
    }
}
```

```
pos = j;
}
}
frames[pos] = pages[i];
count++;
time[pos] = count;
fault++;
}
}
printf("Total Page Faults = %d\n", fault);
return 0;
}
```

SHELL SCRIPTING:

```
#!/bin/bash

echo "LRU Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

echo "Reference String: ${pages[@]}"

echo "LRU Algorithm Executed"
```

OUTPUT:

```
(kaliⓀ Anbuchelvan)~  
$ nano Lru.sh  
  
(kaliⓀ Anbuchelvan)~  
$ chmod +x Lru.sh  
  
(kaliⓀ Anbuchelvan)~  
$ bash Lru.sh  
LRU Page Replacement Demonstration  
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2  
LRU Algorithm Executed
```

C) : OPTIMAL PAGE REPLACEMENT

C PROGRAM:

```
#include <stdio.h>  
  
int main()  
{  
    int pages[50], frames[10];  
    int n, f;  
    int i, j, k, pos;  
    int fault = 0;  
    int found;  
    printf("Enter Number of Pages: ");  
    scanf("%d", &n);  
    printf("Enter Reference String:\n");  
    for(i = 0; i < n; i++)  
        scanf("%d", &pages[i]);  
    printf("Enter Number of Frames: ");  
    scanf("%d", &f);
```

```
for(i = 0; i < f; i++)
frames[i] = -1;
for(i = 0; i < n; i++)
{
found = 0;
for(j = 0; j < f; j++)
{
if(frames[j] == pages[i])
{
found = 1;
break;
}
}
if(found == 0)
{
for(j = 0; j < f; j++)
{
int future = 999;
for(k = i + 1; k < n; k++)
{
if(frames[j] == pages[k])
{
future = k;
break;
}
}
if(j == 0 || future > pos)
```

```

{
pos = future;
}
}

frames[0] = pages[i];
fault++;
}
}

printf("Total Page Faults = %d\n", fault);

return 0;
}

```

SHELL SCRIPTING:

```

#!/bin/bash

echo "Optimal Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

echo "Reference String: ${pages[@]}"

echo "Optimal Algorithm Executed"

```

OUTPUT:

```

(kaliⓈ Anbuchelvan)~]
$ nano Optimal.sh

(kaliⓈ Anbuchelvan)~]
$ chmod +x Optimal.sh

(kaliⓈ Anbuchelvan)~]
$ bash Optimal.sh
Optimal Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Optimal Algorithm Executed

```